

INFORME_BASE_DATOS_DOCKER

Informe — Manejo de la base de datos con Docker (S.I.G.I)

Proyecto: Sistema Inteligente de Gestión de Incendios (backend)
Contexto: MySQL 8 en contenedor Docker Compose
Fecha: Mayo 2026

1. Objetivo del informe

Este documento explica **cómo está montada la base de datos** cuando el backend corre con Docker, **cómo conectarse** desde herramientas como MySQL Workbench o DBeaver, y **cómo ejecutar consultas SQL** (SELECT, inspección de tablas, etc.) sin confundir las bases de cada microservicio.

2. Arquitectura de datos en nuestro Docker Compose

2.1 Una instancia MySQL, varias bases lógicas

En el archivo `docker-compose.yml` existe **un solo servicio** llamado `mysql-sigi` (imagen oficial `mysql:8.0`). Dentro de ese motor MySQL conviven **seis bases de datos**, una por dominio de microservicio, siguiendo la idea de *database per service*:

Base de datos	Microservicio que la usa
<code>db_usuario</code>	<code>servicio-usuario</code>
<code>db_reporte</code>	<code>servicio-reporte</code>
<code>db_ubicacion</code>	<code>servicio-ubicacion</code>
<code>db_emergencia</code>	<code>servicio-emergencia</code>
<code>db_recurso</code>	<code>servicio-recurso</code>
<code>db_notificacion</code>	<code>servicio-notificacion</code>

Las bases se crean al **primer arranque** del contenedor mediante el script `docker/mysql-init/01-databases.sql` (montado en `/docker-entrypoint-initdb.d`). Si el volumen de

datos de MySQL ya existía de antes, MySQL **no vuelve a ejecutar** esos scripts; en ese caso las bases ya deben existir o hay que crearlas a mano (ver sección 7).

2.2 Tablas y esquema

Cada microservicio usa **Spring Data JPA** con `spring.jpa.hibernate.ddl-auto: update` (en los `application.yml` de los servicios). Eso significa:

- Al **arrancar** el servicio, Hibernate compara las **entidades Java** con las tablas en la base correspondiente y **crea o altera** tablas para que coincidan.
- **No** estamos versionando el SQL con Flyway en este proyecto académico; las “consultas de definición” (CREATE TABLE) las genera Hibernate en la práctica.

Para **consultas de lectura** (SELECT) o **inspección**, se usa el cliente SQL que prefieras, conectado a la base correcta.

3. Cómo conectarse desde tu computador (fuera de Docker)

Los microservicios **dentro** de Docker se conectan al host `mysql-sigi` y al puerto **3306** (puerto interno del contenedor). **Tú**, desde MySQL Workbench o la terminal de tu Mac, no usas ese nombre: usas **localhost** y el puerto que Docker **publicó** en el host.

En el `docker-compose.yml` actual, el mapeo por defecto es:

```
host del Mac : 13306 → contenedor MySQL : 3306
```

Eso evita chocar con un MySQL que tengas instalado en el Mac en el puerto 3306.

Parámetros típicos de conexión:

Campo	Valor (por defecto en el proyecto)
Host	<code>127.0.0.1</code> o <code>localhost</code>
Puerto	<code>13306</code> (si no cambiaste <code>MYSQL_PUBLISH_PORT</code>)
Usuario	<code>root</code>
Contraseña	La de <code>MYSQL_PASSWORD</code> en tu <code>.env</code> o compose; por defecto root

En Workbench: *Database* → *Manage Connections* → nueva conexión con esos datos → *Test Connection* → *OK*.

Importante: después de conectar, en Workbench debes **elegir el esquema** (doble clic en `db_reporte`, etc.) o en cada pestaña SQL ejecutar `USE db_reporte`; antes de consultar tablas de ese servicio.

4. Cómo ejecutar consultas SQL (ejemplos)

4.1 Elegir la base correcta

Cada tabla vive en **una** base. Si haces `SELECT * FROM reportes` sin haber seleccionado base, MySQL puede dar error o buscar en la base por defecto equivocada.

Ejemplo de sesión ordenada:

```
USE db_reporte;

SHOW TABLES;

DESCRIBE reportes;

SELECT id, usuario_id, estado, prioridad, direccion, fecha_reporte
FROM reportes
ORDER BY fecha_reporte DESC
LIMIT 20;
```

Para usuarios:

```
USE db_usuario;

SELECT id, nombre, email, rol, activo FROM usuarios;
```

Para emergencias:

```
USE db_emergencia;

SELECT id, reporte_id, estado, prioridad, fecha_creacion FROM emergencias;
```

Los **nombres exactos de columnas** pueden variar ligeramente según cómo Hibernate mapeó las entidades (camelCase en Java suele traducirse a snake_case en MySQL si está configurado así). Si algo no coincide, usa `DESCRIBE nombre_tabla;` en la base correcta.

4.2 Consultas desde la terminal (sin Workbench)

Con Docker:

```
docker exec -it mysql-sigi mysql -uroot -p
```

(La contraseña es la misma que `MYSQL_ROOT_PASSWORD`, por defecto `root` en el proyecto.)

Dentro del cliente `mysql` :

```
SHOW DATABASES;  
USE db_recurso;  
SHOW TABLES;  
SELECT * FROM recursos LIMIT 10;
```

Salir con `exit` .

5. Relación entre “consultas” y lo que hace la aplicación

- **La aplicación** (Spring Boot) ejecuta SQL **a través de JPA** cuando llamas a los endpoints (INSERT al crear un reporte, SELECT al listar, etc.). Tú no necesitas ejecutar esos INSERT a mano para que el sistema funcione.
 - **Las consultas manuales** sirven para **revisar** datos, depurar entregas, sacar capturas para el informe o verificar que Hibernate creó las tablas bien.
 - Si borras o alteras datos “a mano” en tablas que la app sigue usando, puedes **dejar el estado incoherente** (por ejemplo un `usuario_id` que no existe en `db_usuario`). En ambiente de prueba está permitido; en producción se usarían transacciones y restricciones más estrictas.
-

6. Puertos y confusión frecuente

- **13306** en el Mac = “puerta” hacia el MySQL del contenedor.
- **3306** dentro de la red Docker = puerto donde MySQL escucha **dentro** del contenedor; los microservicios usan `mysql-sigi:3306` en su URL JDBC.

Si en el `.env` definiste `MYSQL_PUBLISH_PORT=3306` , entonces desde el Mac conectarías por **3306**, pero solo si nada más ocupa ese puerto en tu máquina.

7. Si no ves las bases o las tablas

1. **Contenedor arriba:** `docker ps` y verificar que `mysql-sigi` está *running*.
 2. **Script de init:** solo corre la primera vez con volumen vacío. Si borraste el volumen o es instalación nueva, deberían crearse las seis bases. Si no, ejecuta manualmente el contenido de `docker/mysql-init/01-databases.sql` conectado como root.
 3. **Tablas vacías o sin tablas:** arranca al menos una vez el microservicio que usa esa base (por ejemplo `servicio-reporte`) para que Hibernate ejecute `ddl-auto: update` y materialice las tablas.
-

8. Buenas prácticas y advertencias (nivel curso)

- No subas a Git el archivo `.env` con contraseñas reales.
 - El usuario `root` sin restricciones es **aceptable solo en laboratorio**.
 - Haz **copias** o exportaciones (`mysqldump`) si necesitas conservar datos antes de `docker compose down` con volúmenes que borren datos (depende de cómo esté definido el compose en tu máquina).
 - Para el informe de la asignatura, un pantallazo de Workbench con un `SELECT` sobre `db_reporte.reportes` suele ser evidencia suficiente de que dominas el manejo de la BD en Docker.
-

9. Conclusión

En S.I.G.I, “manejar la base de datos en Docker” significa: **conocer el puerto publicado, conectar con usuario/contraseña del compose, elegir la base `db_*` correcta** según el microservicio y **ejecutar SQL** con Workbench, DBeaver o el cliente `mysql` dentro del contenedor. Las tablas las mantiene alineadas Hibernate; las consultas manuales son sobre todo para **inspección y aprendizaje**, no para reemplazar el flujo normal de la API.

Documento orientado al uso práctico del equipo S.I.G.I — Mayo 2026.